

Documentação Técnica do Projeto FEMa

Comparação de Bases de Interpolação para o Finite Element Machine

July 24, 2026

Contents

1	Introdução	2
1.1	Objetivo científico do projeto	2
1.2	Visão geral da arquitetura	2
2	O núcleo matemático: <code>core/</code>	4
2.1	<code>core/math/distances/</code> — métricas de distância	4
2.2	<code>core/math/neighbor_search/</code> — busca de vizinhos	4
2.3	<code>core/math/basis/</code> — as funções de base	4
2.3.1	<code>parameters.py</code> — o contêiner único de hiperparâmetros	5
2.3.2	<code>base_basis.py</code> — a classe abstrata <code>BaseBasis</code>	5
2.3.3	<code>_compactSupport.py</code> — bases de suporte compacto	5
2.3.4	As 21 bases registradas	6
2.3.5	<code>factory_basis.py</code> — a fábrica <code>Basis</code>	7
2.4	<code>core/models/</code> — os modelos <code>FEMa</code>	7
2.4.1	<code>base_model.py</code> — <code>FEMaBaseModel</code>	7
2.4.2	<code>fema_classifier.py</code> — <code>FEMaClassifier</code>	7
2.4.3	<code>fema_regressor.py</code> — <code>FEMaRegressor</code>	7
3	Camada de dados: <code>datasets/</code>	8
3.1	<code>registry.py</code> — o que existe e onde	8
3.2	<code>loader.py</code> — leitura, split e padronização	8
4	Camada de modelos: <code>models/</code>	9
4.1	<code>base.py</code> — o contrato <code>ModelPlugin</code>	9
4.2	<code>fema.py</code> — o <code>FEMa</code> como (contexto, base)	9
4.3	<code>logreg.py</code> , <code>knn.py</code> , <code>mlp.py</code> — baselines externos	10
5	Métricas: <code>metrics/</code>	12
5.1	<code>contracts.py</code> — o contrato de métrica	12
5.2	<code>classification.py</code> e <code>regression.py</code> — implementações	12
5.3	<code>registry.py</code> — registro central e direção de otimização	12
6	Busca de hiperparâmetros: <code>tuning/</code>	13
6.1	<code>grid_search.py</code> — Grid Search	13
6.2	<code>random_search.py</code> — Random Search	13
6.3	<code>param_space.py</code> — o ponto único de entrada	13
6.4	<code>cv_strategy.py</code> — geração de splits de validação cruzada	13
7	Treino: <code>training/</code>	15
8	Avaliação: <code>evaluation/</code>	16

9 Persistência: <code>persistence/</code>	17
9.1 Escrita atômica	17
9.2 Checkpoint: retomada de execuções interrompidas	17
9.3 Resumo: <code>summary_builder.py</code>	17
10 Orquestração: <code>pipeline/run_model.py</code>	19
10.1 <code>run_from_experiment_file</code> — roteamento de configuração	20
11 Relatórios e comparação de bases: <code>reporting/</code>	21
11.1 <code>compare_bases.py</code> — o módulo ativo	21
11.2 <code>compare_models.py</code> — arquivo legado, não utilizado	21
11.3 <code>plots.py</code> e <code>curves.py</code> — utilitários de visualização gerais	22
12 Utilitários: <code>utils/</code>	23
13 Configuração declarativa: <code>configs/</code>	24
13.1 <code>base.yaml</code> — defaults globais	24
13.2 <code>experiments/*.yaml</code> — um arquivo por experimento	24
14 Ponto de entrada: <code>main.py</code>	26
15 Testes: <code>tests/</code>	27
15.1 <code>test_pipeline.py</code> — testes de fumaça do pipeline	27
15.2 <code>test_reporting.py</code> — testes do módulo de comparação	27
15.3 <code>tests/linear_algebra/</code> — validação matemática das bases	27
16 Código legado e pendências	29
17 Como expandir o projeto	30
17.1 Adicionando um novo dataset	30
17.2 Adicionando uma nova base de interpolação ao FEMa	30
17.3 Habilitando análise de regressão	31
17.4 Comparando com outros modelos (baselines)	32
17.4.1 Re-habilitar um baseline já implementado (MLP)	32
17.4.2 Adicionar um baseline totalmente novo	33
17.5 Criando um novo experimento (YAML)	33
17.6 Adicionando uma nova métrica	34
17.7 Checklist resumo	34
18 Conclusão	35

1. Introdução

1.1 Objetivo científico do projeto

Este projeto implementa e avalia o **FEMa** (*Finite Element Machine*), um método de classificação/regressão baseado em interpolação por vizinhos próximos: para prever o rótulo (ou valor) de uma amostra nova, o FEMa busca os k vizinhos mais próximos no conjunto de treino e combina os valores desses vizinhos usando pesos derivados de uma **função de base** $\varphi(d)$, onde d é a distância até cada vizinho.

O objeto de estudo central do projeto **não é o FEMa em si**, mas sim **a comparação entre as diferentes funções de base** que podem ser usadas dentro dele — Shepard, RBF Gaussiana, Multiquadrática, Wendland C2, splines, kernels de Cauchy, Student-t, Racional Quadrático, entre outras (21 ao todo, listadas na Seção 2.3). A pergunta de pesquisa é, essencialmente: *dado um dataset e uma tarefa (classificação ou regressão), qual função de base produz o melhor desempenho preditivo, e por quê?*

Essa é a razão pela qual, como será detalhado ao longo deste documento, toda a arquitetura do código — do desenho dos *plugins* de modelo à organização dos resultados em disco — foi construída para que a **base de interpolação seja o eixo estrutural de comparação**, e não apenas mais um hiperparâmetro entre outros. Baselines externos (regressão logística, k-NN) existem apenas como *referência metodológica* — para responder “o FEMa com a melhor base é competitivo com um classificador simples?” — e ficam deliberadamente isolados do restante da comparação.

1.2 Visão geral da arquitetura

O projeto segue uma arquitetura em camadas, na qual cada diretório do repositório tem uma responsabilidade única e bem delimitada. A regra geral é: **uma camada só conhece a camada imediatamente abaixo dela**. Isso significa, por exemplo, que a camada de persistência (*persistence/*) não sabe o que é uma “base” ou um “dataset” — ela só grava e lê JSON em caminhos genéricos; quem decide o *significado* de cada segmento do caminho é a camada de orquestração (*pipeline/*).

O fluxo de dependências entre camadas, da base para o topo, resumido a seguir:

1. **core/** — o núcleo matemático do FEMa (bases, distâncias, busca de vizinhos, os modelos *FEMaClassifier/FEMaRegressor*). É tratado como uma *biblioteca fechada*: nenhuma decisão de experimento deveria exigir alterá-lo.
2. **datasets/** — carregamento e split de dados, desacoplado de qualquer modelo.
3. **models/** — a camada de *plugins*: adapta o FEMa (*core/*) e os baselines externos (scikit-learn) a um contrato único (*ModelPlugin*).

4. **metrics/**, **tuning/**, **training/**, **evaluation/** — blocos de construção independentes usados pelo orquestrador: cálculo de métricas, geração de combinações de hiperparâmetros, treino e avaliação.
5. **persistence/** — escrita atômica dos resultados em JSON e sistema de checkpoint (retomada de execuções interrompidas).
6. **pipeline/run_model.py** — o orquestrador único que liga todas as camadas acima em um fluxo completo: carregar dados → checkpoint → busca de hiperparâmetros → validação cruzada → resumo → retreino final → avaliação no teste.
7. **reporting/** — lê os resultados já persistidos e produz tabelas, JSON consolidado e gráficos comparando as bases entre si.
8. **utils/** — utilitários transversais (log, seed, tempo, hash).
9. **configs/** — arquivos YAML declarativos que parametrizam experimentos, sem exigir alteração de código.
10. **main.py** — ponto de entrada de linha de comando.

Os capítulos a seguir documentam cada camada nessa mesma ordem — da matemática para a orquestração —, e o Capítulo 17 explica concretamente como estender o projeto em cada uma das direções mais prováveis: novos datasets, novas bases, análise de regressão e comparação com outros modelos.

2. O núcleo matemático: `core/`

`core/` contém tudo o que é **matemática do FEMa em si** — as funções de base, as métricas de distância, a estrutura de busca de vizinhos e os dois modelos (`FEMaClassifier`, `FEMaRegressor`). Por convenção do projeto, esta pasta é tratada como uma biblioteca estável: decisões de *experimento* (que faixa de hiperparâmetros testar, quais métricas calcular, como organizar os resultados) vivem fora dela, nas camadas superiores.

2.1 `core/math/distances/` — métricas de distância

Três arquivos pequenos definem o contrato de distância:

- `base_distance.py`: classe abstrata `BaseDistance` com um único método, `compute(x1, x2)`, vetorizado (aceita tanto um par de vetores quanto um vetor contra uma matriz de N vetores).
- `euclidean_distance.py`: `EuclideanDistance`, a métrica usada em praticamente todo o projeto — norma L_2 da diferença, com `np.linalg.norm`.
- `manhattan_distance.py`: `ManhattanDistance`, norma L_1 , disponível mas não usada em nenhum experimento atual (ver Seção 17.4 sobre como conectá-la).

2.2 `core/math/neighbor_search/` — busca de vizinhos

`BaseSearch` define o contrato: `build(X)` indexa o conjunto de treino, e `query(sample, k)` retorna os índices e distâncias dos k vizinhos mais próximos de uma amostra (ou de todos, se $k = 0$).

A única implementação existente, `BruteForceSearch` (`brute_force.py`), calcula a distância de `sample` contra **todos** os pontos de treino e usa `np.argpartition` para selecionar os k menores em tempo $O(n)$ (sem ordenar o vetor inteiro). É deliberadamente simples: para os tamanhos de dataset usados no projeto, uma estrutura de indexação espacial (KD-tree, ball-tree) traria complexidade adicional sem necessidade prática. Trocar a estratégia de busca por uma mais eficiente é uma extensão natural e isolada (basta implementar `BaseSearch` — ver Seção 17.4).

2.3 `core/math/basis/` — as funções de base

Esta é a parte mais extensa e mais importante de `core/`, pois **as funções de base são o objeto de estudo do projeto**.

2.3.1 `parameters.py` — o contêiner único de hiperparâmetros

Antes de qualquer base específica, vale entender `BasisParameters` (`parameters.py`): um *dataclass* único com nove campos opcionais (`z`, `epsilon`, `c`, `alpha`, `beta`, `nu`, `h`, `l`, `p`), todos com valor-padrão `None`. Essa é uma decisão de design central: em vez de cada base ter sua própria assinatura de função (o que obrigaria quem chama a base — o classificador, o harness de validação, a camada de tuning — a conhecer a fórmula de cada uma para montar os argumentos certos), **toda base recebe a mesma assinatura** `compute_weights(dists, params)`, e cada implementação lê, de `params`, apenas os campos que sua fórmula usa. Um campo faz o papel de vários símbolos matemáticos diferentes dependendo da base (por exemplo, `c` é usado por Multiquadrática, Inversa Multiquadrática, Logarítmica e Sigmoidal, cada uma com um papel distinto na fórmula) — isso é intencional: o nome é só o “slot” de configuração, não implica semântica compartilhada.

2.3.2 `base_basis.py` — a classe abstrata `BaseBasis`

`BaseBasis` separa duas camadas deliberadamente:

- `evaluate(dists, params)`: o valor **cru** (não normalizado) de $\varphi(d)$. Cada subclasse concreta implementa apenas isto — a fórmula da base em si.
- `compute_weights(dists, params)`: o peso **normalizado** (partição da unidade), $w = \varphi(d) / \sum \varphi(d)$. Implementado uma única vez, na classe base, em cima de `evaluate()`. Nenhuma subclasse deveria reimplementar isso.

Essa separação corrige, por construção, uma classe inteira de bug que existia em versões anteriores do código: a normalização feita incorretamente dentro de cada base individualmente (nos comentários do projeto, “`weights /= weights`” em vez de “`weights /= weights.sum()`”). Centralizar a normalização em `BaseBasis` elimina essa possibilidade de erro para qualquer base nova.

Além disso, se $\sum \varphi(d) = 0$ (uma base de suporte compacto em que nenhum vizinho cai dentro do raio, por exemplo), `compute_weights` recorre a pesos uniformes em vez de propagar um NaN silenciosamente.

Cada subclasse declara dois atributos de classe:

- `PARAMS`: os nomes dos campos de `BasisParameters` que a fórmula **exige**. O método `_require(params)` extrai esses campos e levanta um `ValueError` claro se algum estiver `None`, em vez de deixar o erro estourar mais adiante como uma exceção aritmética difícil de rastrear.
- `OPTIONAL_PARAMS`: campos que a base usa, mas que são opcionais (o caso mais comum é `h` nas bases de suporte compacto, onde `None` significa “raio automático, igual à maior distância observada”).

2.3.3 `_compactSupport.py` — bases de suporte compacto

`CompactSupportBasis` é uma classe intermediária, herdada por `Wendland C2`, `Cubic Spline`, `Quartic Spline` e `Cosine`. Fornece `_normalized_distance(dists, h)`, que calcula $r = d/h$ — usando $h = \max(d)$ automaticamente quando `h` não é informado. Fora do intervalo $r \leq 1$, a base vale zero: são funções de **suporte compacto**, que ignoram vizinhos além de um certo raio.

2.3.4 As 21 bases registradas

A Tabela 2.1 resume as 21 funções de base atualmente implementadas, todas em `core/math/basis/`, registradas na fábrica `Basis` (`factory_basis.py`, Seção 2.3.5). Note que várias bases têm notas de equivalência matemática documentadas no próprio código-fonte (docstrings marcadas “NOTA”): decisões de nomenclatura e de fórmula ainda pendentes de confirmação com o orientador, deixadas explícitas em vez de escondidas.

Table 2.1: As 21 funções de base registradas em `Basis` (`factory_basis.py`).

Nome istry)	(reg- istry)	Fórmula $\varphi(d)$	Parâmetros
shepard		$1/d^z$	z
radial		$\exp(-0.5(d/r_0)^2)$, $r_0 = z$	z
rbf_gaussian		$\exp(-(\epsilon d)^2)$	epsilon
multiquadratic		$\sqrt{d^2 + c^2}$ (crescente em d)	c
inverse_multiquadratic		$1/\sqrt{d^2 + c^2}$	c
wendland_c2		$(1-r)^4(4r+1)$, $r = d/h$, 0 se $r > 1$	h (opc.)
cubic_spline		$(1-r)^4$, $r = d/h$, 0 se $r > 1$	h (opc.)
quartic_spline		$(1-r)^4$, $r = d/h$, 0 se $r > 1$ (idêntica a <code>cubic_spline</code> — nota de decisão pendente)	h (opc.)
gen_exponential		$\exp(-\epsilon d^p)$	epsilon, p
softmax_radial		$\exp(-\beta d)$ (nota: equivalente a <code>laplacian</code> após normalização)	beta
attention		$1/(1+d^2)$ (nota: equivalente a <code>cauchy</code> com $\epsilon = 1$)	—
logarithmic		$1/\log(1+d+c)$	c
harmonic		$(1+d^2/\nu)^{-(\nu+1)/2}$ (nota: formalmente idêntica a <code>student_t</code>)	nu
laplacian		$\exp(-\epsilon d)$	epsilon
cauchy		$1/(1+(\epsilon d)^2)$	epsilon
student_t		$(1+d^2/\nu)^{-(\nu+1)/2}$ (kernel-t, van der Maaten & Hinton, 2008)	nu
cosine		$\cos^2(\pi r/2)$, $r = d/h$, 0 se $r > 1$	h (opc.)
sigmoidal		$1/(1+\exp(\alpha(d-c)))$	alpha, c
lorentzian		$1/(1+d^4)$ (nota: na literatura “Lorentzian” costuma ser sinônimo de Cauchy)	—
entropic		$\exp(-\beta d^2)$	beta
rational_quadratic		$1/(1+d^2/(2\alpha^2))^{-\alpha}$ (Rasmussen & Williams, 2006)	alpha, l

Todas seguem o mesmo padrão de arquivo — pequeno, uma classe, um método `evaluate`:

```
class ShepardBasis(BaseBasis):
    PARAMS = ("z",)

    def evaluate(self, dists, params):
        z = self._require(params)["z"]
        dists = np.where(dists == 0, DELTA, dists) # evita divisao
            por zero
        return 1.0 / (dists ** z)
```

2.3.5 `factory_basis.py` — a fábrica `Basis`

`Basis` é o ponto único de acesso às implementações: `Basis.get(nome, search)` instancia a base pelo nome (string), e `Basis.available()` retorna a lista ordenada de todos os nomes registrados no dicionário interno `_CLASSES`. Essa função é usada extensivamente por outras camadas (o harness de validação matemática, os *plugins* de modelo, o módulo de comparação de bases) para **descobrir automaticamente** quais bases existem, sem duplicar essa lista em vários lugares do projeto — uma base nova só precisa de uma entrada em `_CLASSES` para se tornar visível a todo o resto do sistema (ver Seção 17.2).

2.4 `core/models/` — os modelos `FEMa`

2.4.1 `base_model.py` — `FEMaBaseModel`

Classe abstrata que define o contrato comum: o modelo guarda `X_train/y_train`, delega a indexação ao `search` injetado no construtor, e implementa `fit/predict` como métodos abstratos. A ideia de responsabilidade é: *o modelo orquestra search + basis e faz o produto interno final; a busca de vizinhos e o cálculo de pesos são delegados*.

2.4.2 `fema_classifier.py` — `FEMaClassifier`

Para classificação, o `fit` constrói uma matriz de probabilidades por classe: uma linha por classe, com 1.0 nas posições de treino cuja classe é aquela e 0.0 nas demais (codificação one-hot transposta). O `predict`, para cada amostra de teste, busca os k vizinhos uma única vez, calcula os pesos com a base escolhida, e então interpola a probabilidade de **cada** classe usando os **mesmos** pesos (produto interno entre o vetor de pesos e a linha da classe). O rótulo previsto é o `argmax` das probabilidades interpoladas.

2.4.3 `fema_regressor.py` — `FEMaRegressor`

Para regressão, o processo é mais direto: o `fit` apenas guarda `X_train/y_train` e indexa `X_train` no `search`; o `predict` interpola diretamente o valor-alvo (produto interno entre pesos e `y_train[indices]`), com uma salvaguarda para o caso raro em que a predição resulte em NaN (cai para a média de `y_train`).

Este componente já existe e está pronto para uso — ver Seção 17.3 sobre como habilitar experimentos de regressão de ponta a ponta.

3. Camada de dados: datasets/

3.1 registry.py — o que existe e onde

Define `DatasetSpec`, um *dataclass* imutável com o caminho do CSV, a coluna-alvo, as colunas a descartar e o delimitador. O dicionário `_REGISTRY` mapeia um nome lógico de dataset para essa especificação. Quatro datasets estão registrados hoje:

- `fetal_health` — CSV real, alvo `fetal_health` (3 classes).
- `iris` — CSV real, alvo `Species`, descarta a coluna `Id`.
- `classification_data` — CSV real com delimitador `;`, alvo `class`.
- `synthetic_demo` — não tem arquivo; é gerado em tempo de execução (ver `loader.py`) — usado nos testes automatizados, por ser rápido e não depender de arquivos externos.

Este módulo **nunca lê nem transforma dados** — apenas descreve onde/como encontrá-los. A função `get_dataset_spec(name)` valida que o arquivo existe no disco antes de devolver a especificação (levanta `FileNotFoundError` cedo, com mensagem clara, em vez de falhar mais adiante dentro do *pandas*).

3.2 loader.py — leitura, split e padronização

`load_dataset(dataset_name, seed, val_ratio, test_ratio, scale)` é a única função pública relevante. Faz, nesta ordem:

1. Lê o CSV (via `pandas.read_csv`, normalizando nomes de coluna e removendo o BOM `\ufeff` que alguns arquivos trazem) ou gera o dataset sintético (`sklearn.datasets.make_classification`).
2. Separa `X` (todas as colunas menos o alvo, convertidas para `float64`) de `y` (convertido para inteiro diretamente se já for numérico, ou por `codes` de categoria caso contrário).
3. Faz o split treino/validação/teste em duas chamadas encadeadas de `train_test_split` (ambas `stratify`), de forma **determinística** (mesma `seed` sempre produz o mesmo split).
4. Se `scale=True` (padrão), padroniza todas as partições ($x' = (x - \mu)/\sigma$) usando **apenas** estatísticas do conjunto de treino — esta padronização é essencial para o FEMa, que é um método baseado em distância: sem ela, variáveis em escalas diferentes dominariam artificialmente o cálculo de distância.

O retorno é um `LoadedData` (*dataclass* imutável) com `X_train/y_train/X_val/y_val/X_test/y_test` e o nome do dataset. Esta camada não conhece nenhum modelo nem faz parte do pipeline de tuning/avaliação — é puramente sobre dados.

4. Camada de modelos: `models/`

4.1 `base.py` — o contrato `ModelPlugin`

Esta é a peça de desenho mais importante do projeto depois de `core/`: **nenhuma outra camada importa um algoritmo diretamente**. Tudo — treino, avaliação, tuning, persistência — fala apenas com a interface abstrata `ModelPlugin`. Isso é o que permite trocar de modelo (ou adicionar um novo) sem tocar em `pipeline/`, `evaluation/`, `training/` ou `tuning/`.

Os métodos do contrato:

- `create_model(params, random_state)` (*abstrato*): instancia (sem treinar) um estimador com os hiperparâmetros dados.
- `parameter_grid()` (*abstrato*): espaço de busca **discreto**, para `grid_search` — formato `{nome: [valores]}`.
- `random_search_space()` (*opcional, com default = reaproveitar `parameter_grid()`*): espaço de busca **contínuo**, para `random_search` — aceita também tuplas ("uniform", low, high), ("loguniform", low, high) e ("randint", low, high). Esta é uma extensão em relação ao projeto de referência que inspirou esta arquitetura (que só tinha grades discretas): necessária porque o FEMa e os baselines aqui têm hiperparâmetros contínuos (z, epsilon, C, ...) mal representados por uma grade fixa.
- `fit(estimator, X, y, X_val, y_val)`: por padrão, chama `estimator.fit(X, y)`; plugins podem sobrescrever se precisarem de `X_val/y_val` (ex.: *early stopping*).
- `predict/predict_proba`: delegam ao estimador; `predict_proba` retorna `None` graciosamente se o modelo não suportar (o cálculo de métricas trata isso sem quebrar, pulando métricas que dependem de probabilidade, como `roc_auc`).
- `save/load`: serialização via `joblib`.

4.2 `fema.py` — o FEMa como (contexto, base)

Este é o arquivo que mais evoluiu ao longo do desenvolvimento do projeto, e a evolução em si é instrutiva. Numa versão anterior, existia um único `FEMaPlugin` “genérico”, e a base de interpolação era apenas mais um valor dentro do espaço de hiperparâmetros buscado (`basis_function`), junto com `k`, `z`, `epsilon` etc. Isso tinha um problema conceitual sério: o *ranking* final do resumo (`persistence/summary_builder.py`) acabava misturando combinações de bases **diferentes** na mesma lista, de modo que a “melhor combinação” encontrada respondia à pergunta errada (“qual é a melhor combinação global entre todas as bases misturadas”, em vez de “qual é o melhor desempenho *de cada* base”).

A arquitetura atual resolve isso fixando o **contexto** ("classifier" ou "regressor") e a

base na própria instância do plugin:

```
class FEMaPlugin(ModelPlugin):
    def __init__(self, context: str, basis: str):
        ...
        self.context = context
        self.basis = basis
        self.name = f"fema:{context}:{basis}"
        self.supports_proba = context == "classifier"

    def parameter_grid(self) -> Dict[str, list]:
        return {"k": _K_GRID, **PARAM_GRIDS[self.basis]}
```

Consequências desse desenho:

- Cada execução do pipeline já é, por construção, escopada a uma única base — não existe mais a possibilidade de misturar bases no mesmo resumo.
- `parameter_grid()/random_search_space()` voltam a ser um único `Dict[str, list]` simples (`k` + os parâmetros **próprios** daquela base), sem precisar de um espaço de busca condicional/ramificado.
- `PARAM_GRIDS/RANDOM_SEARCH_SPACES` — os dicionários que mapeiam nome de base para a faixa de busca de cada parâmetro — continuam centralizados aqui, e não dentro de `core/math/basis/`, por decisão deliberada: são faixas de busca de **experimento** (decisão de quem desenha o experimento), não uma propriedade matemática da base (que pertence a `core/`).
- `_validate_basis_entry(basis_name)` confere, na **construção** de cada `FEMaPlugin`, que `PARAM_GRIDS/RANDOM_SEARCH_SPACES` cobrem **exatamente** os campos que `basis.PARAMS` | `basis.OPTIONAL_PARAMS` declara — nem a mais, nem a menos. Isso significa que registrar uma base nova em `core/` sem adicionar a entrada correspondente aqui produz um erro imediato e claro, no momento de instanciar o plugin, em vez de uma falha silenciosa ou um espaço de busca incompleto no meio de um tuning de horas.

O adaptador `_FEMaEstimator` é o que dá ao FEMa uma interface “*sklearn-like*” (`fit/predict/predict_proba`) para caber no contrato `ModelPlugin`: ele escolhe `FEMaClassifier` ou `FEMaRegressor` conforme o `context`, instancia a base via `Basis.get`, e traduz `predict` de acordo (um classificador retorna (`labels`, `probs`); um regressor retorna só o vetor de previsões).

A função `available_bases()` apenas repassa `Basis.available()` — usada por `pipeline.run_model.run_all` e por `reporting/compare_bases.py` para nunca duplicar essa lista em outro lugar do projeto.

4.3 logreg.py, knn.py, mlp.py — baselines externos

`models/registry.py` deixa claro, na própria docstring, que **o FEMa não está neste registro**: ele não é identificado por um único nome, mas por um par (`context`, `basis`) (ver `FEMaPlugin` acima), então não cabe no formato “nome → instância fixa” usado para baselines. Este registro existe apenas para os baselines **externos**, que servem de referência metodológica e não fazem parte da comparação de bases:

- `LogRegPlugin` (`logreg.py`) — regressão logística do scikit-learn, com cuidado explícito de só passar `penalty/solver` quando diferentes do padrão (evita um `FutureWarning` de depreciação em versões recentes do scikit-learn).
- `KNNPlugin` (`knn.py`) — k-Vizinhos Mais Próximos (`KNeighborsClassifier`). É o base-

line mais metodologicamente próximo do FEMa (também baseado em vizinhança), o que o torna uma referência particularmente informativa: mostra o quanto a *escolha da base de interpolação* agrega sobre uma vizinhança “crua”.

- `MLPPlugin (mlp.py)` — uma rede neural simples (`MLPClassifier`) **existe como arquivo**, mas **não está registrada** em `models/registry.py::_PLUGINS` na versão atual do código. Ou seja, o arquivo está pronto e funcional, mas inacessível via `model_name="mlp"` até que alguém a adicione de volta ao dicionário `_PLUGINS` — um único ponto de reabilitação (ver Seção 17.4).

Ambos os baselines registrados (`logreg`, `knn`) seguem exatamente o mesmo padrão: `parameter_grid()` para grade discreta, `random_search_space()` para distribuições contínuas, sem nenhuma lógica além da tradução direta para os construtores do scikit-learn.

5. Métricas: `metrics/`

5.1 `contracts.py` — o contrato de métrica

Toda métrica é uma função com a assinatura `metric(y_true, y_pred, y_score, n_classes)` -> `Optional[float]`. `y_score` é a matriz de probabilidades (ou `None`, se indisponível); `n_classes` é usado por métricas que se comportam diferente em binário vs. multiclasse. Retornar `None` (em vez de levantar exceção) é o sinal esperado para “não calculável nesta situação” — por exemplo, `roc_auc` sem `y_score`, ou quando um fold contém uma única classe.

5.2 `classification.py` e `regression.py` — implementações

Sete métricas de classificação (via `sklearn.metrics`): `accuracy`, `balanced_accuracy`, `precision`, `recall`, `f1`, `roc_auc`, `mcc`. Todas são “*multiclass-aware*”: quando há mais de duas classes, usam `average='macro'` (não ponderado por suporte, para não mascarar desempenho ruim em classes minoritárias) e, no caso de `roc_auc`, `multi_class='ovr'`.

Cinco métricas de regressão: `mae`, `mse`, `rmse`, `r2`, `mape`. Seguem o **mesmo contrato** (recebem `y_score/n_classes`, mas os ignoram) — isso é o que permite que o mesmo motor de avaliação (`compute_all`) sirva tanto para classificação quanto para regressão sem nenhuma ramificação de código.

5.3 `registry.py` — registro central e direção de otimização

`_METRICS` mapeia nome -> função; `HIGHER_IS_BETTER` mapeia nome -> booleano (usado por `persistence/summary_builder.py` para saber se, ao rankear pela métrica primária, um valor maior é melhor — caso de `f1` — ou pior — caso de `mae`). Duas listas adicionais, `CLASSIFICATION_METRICS` e `REGRESSION_METRICS`, são a fonte única de verdade para “todas as métricas registradas” de cada tipo de tarefa, acessadas através de `metrics_for_context(context)`. Isso é usado por `pipeline/run_model.py` para decidir os *defaults* de `metric_names` conforme o contexto (classificador -> 7 métricas; regressor -> 5 métricas), e por `reporting/compare_bases.py` e pelos testes para garantir que a comparação cobre **todas** as métricas relevantes, e não um subconjunto escolhido à mão em cada lugar que precisa dessa lista.

`compute_all(metric_names, y_true, y_pred, y_score, n_classes)` calcula todas as métricas pedidas de uma vez, capturando qualquer exceção individual e gravando `None` sem interromper as demais.

6. Busca de hiperparâmetros: `tuning/`

6.1 `grid_search.py` — Grid Search

`expand_param_grid(param_grid)` expande um dicionário `{chave: [valores]}` no produto cartesiano de todas as combinações. Também suporta, por compatibilidade histórica, `List[Dict[str, list]]` — uma “ramificação” por dicionário, útil quando um hiperparâmetro só faz sentido para certos valores de outro (o caso que motivou originalmente esse formato foi o antigo `FEMaPlugin` genérico, descrito na Seção 4.2; hoje nenhum plugin do projeto usa mais esse formato condicional, mas o suporte permanece disponível para um plugin futuro que precise dele). `combo_id(params)` gera um identificador estável e legível (string ordenada por chave), usado como chave de checkpoint e no ranking do resumo.

6.2 `random_search.py` — Random Search

`sample_param_space(search_space, n_iter, seed)` amostra `n_iter` combinações de um espaço de busca contínuo, de forma determinística dada a `seed`. Suporta os três tipos de distribuição descritos na Seção 4 (`uniform`, `loguniform`, `randint`) e também listas de escolha discreta. Assim como `grid_search.py`, aceita o formato condicional `List[Dict]` por compatibilidade — no caso condicional, cada amostra sorteia primeiro **qual** ramificação usar (peso igual entre elas) e só então amostra os campos daquela ramificação, nunca misturando campos de ramificações diferentes numa mesma amostra.

6.3 `param_space.py` — o ponto único de entrada

`generate_combinations(plugin, strategy, seed, n_iter, fixed_hyperparameters)` é a única função que `pipeline/run_model.py` chama: decide, com base em `strategy` (`"grid_search"` ou `"random_search"`), qual dos dois motores usar. Se `fixed_hyperparameters` for informado, ignora completamente a estratégia de tuning e usa exatamente aquela combinação — útil para reproduzir um resultado específico ou rodar sem busca de hiperparâmetros (por exemplo, ao gerar uma curva de sensibilidade a um único parâmetro mantendo os demais fixos).

6.4 `cv_strategy.py` — geração de splits de validação cruzada

`build_cv_splits(X, y, strategy, n_splits, n_repeats, master_seed)` é um gerador que produz tuplas (`repeat_idx`, `fold_idx`, `train_idx`, `val_idx`). Duas estratégias: `stratified_kfold` (uma única passada) e `repeated_stratified_kfold` (repete com seeds diferentes, reduzindo a variância da estimativa — útil com datasets pequenos ou desbalanceados). *Nested CV* não é oferecido de propósito: a separação treino/validação (usada aqui para o tuning) e o *holdout* de teste (Capítulo 8) já cumprem o mesmo papel metodológico sem o custo computacional

multiplicado de uma validação cruzada aninhada. A seed do particionador é sempre **derivada** da seed mestre (`utils.seeding.derive_seed`), garantindo que o mesmo conjunto de splits seja gerado sempre que o pipeline for reexecutado com a mesma configuração — condição necessária para o checkpoint funcionar corretamente (Capítulo 9).

7. Treino: `training/`

Dois arquivos pequenos e sem lógica de negócio própria — só orquestram chamadas ao `ModelPlugin`:

- `trainer.py::train_estimator(plugin, params, X_train, y_train, random_state, X_val, y_val)` — instancia (`plugin.create_model`) e treina (`plugin.fit`) um estimador. Não sabe nada sobre validação cruzada, busca de hiperparâmetros, métricas ou persistência.
- `final_fit.py::fit_final_model(plugin, best_params, X_train, y_train, master_seed, X_val, y_val)` — chamado **depois** que o tuning + CV determina os melhores hiperparâmetros (Capítulo 9), treina um modelo **novo do zero** (não reaproveita nenhum modelo de fold), usando uma seed derivada especificamente para o retreino final (`derive_seed(master_seed, "final_fit", plugin.name)`). Este é o único modelo que será avaliado no conjunto de teste.

8. Avaliação: `evaluation/`

- `cv_evaluator.py::evaluate_fold(plugin, params, X_train_fold, y_train_fold, X_val_fold, y_val_fold, metric_names, random_state)` — treina no split de treino do fold, avalia no split de validação, retorna as métricas calculadas. Não sabe nada sobre persistência.
- `test_evaluator.py::evaluate_on_test(plugin, estimator, X_test, y_test, y_train_reference, metric_names)` — chamado **uma única vez**, depois do retreino final. O conjunto de teste nunca é usado antes deste ponto — nem no grid/random search, nem na validação cruzada, nem em nenhuma decisão de tuning. Isso é o que garante que a métrica de teste reportada é uma estimativa honesta de generalização, e não um número inflado por vazamento de informação do teste para o tuning.

Ambos delegam o cálculo efetivo das métricas a `metrics.registry.compute_all` (Capítulo anterior) — a única diferença entre eles é *quando* e *sobre qual partição de dados* são chamados.

9. Persistência: `persistence/`

Esta camada sofreu uma mudança arquitetural deliberada e importante: **deixou de saber o que é “modelo” ou “experimento”**. As três funções centrais de `run_writer.py` (`next_run_path`, `load_all_runs`, `write_summary`, `write_test_results`) recebem um número arbitrário de segmentos de caminho, `*scope`, e simplesmente os concatenam sob `results/`:

```
def _run_dir(*scope: str) -> Path:
    d = RESULTS_DIR.joinpath(*scope)
    d.mkdir(parents=True, exist_ok=True)
    return d
```

O **significado** de cada segmento — se é um contexto, uma base, um dataset, um nome de experimento, ou o nome de um baseline — é responsabilidade exclusiva de quem monta o caminho (`pipeline/run_model.py`, Capítulo 10), nunca da camada de persistência. Essa inversão de dependência é o que permite que `results/` seja organizado por **base** (o eixo científico do projeto) sem que `persistence/` precise ter qualquer conhecimento sobre o que é uma “base”.

9.1 Escrita atômica

`write_run_atomic(path, payload)` grava em um arquivo temporário no **mesmo** diretório de destino e só então faz os `replace` (rename atômico ao nível do sistema operacional). Isso garante que nunca existirá, em disco, um JSON parcialmente escrito — mesmo que o processo seja interrompido (`Ctrl+C`, queda de energia, *out-of-memory*) no meio da escrita.

9.2 Checkpoint: retomada de execuções interrompidas

`checkpoint.py::get_completed_keys(*scope)` varre todos os `run_*.json` já existentes em um diretório de escopo e reconstrói o conjunto de chaves (`combo_id`, `repeat_idx`, `fold_idx`) já executadas. `is_done(completed_keys, combo_id, repeat_idx, fold_idx)` é a checagem usada, fold a fold, dentro do laço principal do pipeline (Capítulo 10), para pular o que já foi feito. Isso é o que torna o pipeline **idempotente**: chamá-lo duas vezes com os mesmos argumentos não duplica trabalho nem resultados — a segunda chamada simplesmente constata que tudo já está pronto e vai direto para o resumo.

9.3 Resumo: `summary_builder.py`

`build_summary(*scope, primary_metric, higher_is_better, extra_fields)` lê todos os `run_*.json` de um escopo, agrupa por `combo_id`, calcula média e desvio-padrão de cada métrica entre folds/repetições, e ordena as combinações pela métrica primária (respeitando a

direção de otimização — Capítulo anterior). O resultado (`summary.json`) inclui `best_configuration` (a melhor combinação encontrada) e o `ranking` completo.

Como consequência direta da mudança para `FEMaPlugin(context, basis)` fixo (Seção 4.2), **cada diretório de resultados corresponde hoje a uma única base do FEMa** (ou a um único baseline externo) — logo, o ranking dentro de um `summary.json` é sempre **homogêneo**: só varia `k` e os parâmetros próprios daquela base. A comparação *entre* bases diferentes é responsabilidade de `reporting/` (Capítulo 11), que lê um `summary.json` por base.

`extra_fields` é injetado no resumo exatamente como veio de quem chamou (por exemplo, `{"context": "classifier", "basis": "shepard", "experiment": "baseline"}`) — `summary_builder` não precisa interpretar o que esses campos significam, apenas propagá-los para o JSON final.

10. Orquestração: `pipeline/run_model.py`

Este é o módulo que **liga todas as camadas anteriores**. Reflete, na própria API pública, a divisão conceitual do projeto entre dois tipos de experimento com objetivos completamente diferentes:

- `run_basis_experiment(context, basis, dataset, ...)` — roda o FEMa com o contexto e a base fixos. É o pipeline **central** da pesquisa: cada chamada corresponde a uma célula da matriz (contexto $\times$ base) sendo avaliada. Persiste em `results/<context>/<basis>/<dataset>`.
- `run_all_bases(context, dataset, bases=None, ...)` — chama `run_basis_experiment` para **todas** as bases registradas em `core.Basis.available()` (ou para um subconjunto, se `bases` for informado). É o modo de uso mais comum do projeto: comparar todas as bases de uma vez, para um mesmo contexto/dataset.
- `run_baseline_experiment(model_name, dataset, ...)` — roda um baseline externo (logreg, knn). **Não** faz parte da comparação de bases; serve apenas de referência metodológica (“o FEMa com a melhor base é competitivo com um classificador simples?”). Persiste, isolado, em `results/external_baselines/<model_name>/<dataset>/<experiment>`.

Os três compartilham o mesmo motor interno, `_execute(scope, plugin, dataset, ...)`, que implementa o fluxo completo:

1. Fixa a seed global (`utils.seeding.set_global_seed`).
2. Carrega o dataset (`datasets.loader.load_dataset`).
3. Gera as combinações de hiperparâmetros a avaliar (`tuning.param_space.generate_combinations`).
4. Consulta o checkpoint (`persistence.checkpoint.get_completed_keys`) para saber o que já foi feito.
5. Para cada combinação $\times$ cada fold de validação cruzada ainda não executado: treina e avalia (`evaluation.cv_evaluator.evaluate_fold`), mede o tempo de execução (`utils.timing.timer`) e grava o resultado imediatamente (`persistence.run_writer.write_run_atomic`) — nunca espera o fim de tudo para persistir nada.
6. Constrói o resumo (`persistence.summary_builder.build_summary`), que identifica a melhor combinação de hiperparâmetros.
7. Retreina um modelo final com essa combinação, no conjunto de treino completo (`training.final_fit.f`).
8. Avalia esse modelo final, uma única vez, no conjunto de teste (`evaluation.test_evaluator.evaluate_o`) e grava o resultado (`persistence.run_writer.write_test_results`).

O motor **não sabe** se `plugin` é o FEMa ou um baseline externo — fala apenas com o contrato `ModelPlugin` (Capítulo 4). A diferença entre `run_basis_experiment` e `run_baseline_experiment` está inteiramente em *como cada uma monta o `scope` (caminho de resultados) e os `metric_names/ranking_metr` padrão antes de chamar `_execute`*.

Note, em particular, que `run_basis_experiment` escolhe as métricas e a métrica de *ranking* padrão de acordo com o contexto:

```
is_classifier = context == "classifier"
default_metrics = DEFAULT_CLASSIFICATION_METRICS if is_classifier else
    DEFAULT_REGRESSION_METRICS
default_ranking = "f1" if is_classifier else "rmse"
```

Isso é o que torna a análise de regressão (Seção 17.3) uma questão de *configuração*, e não de código novo: bastam um dataset com alvo contínuo e `context="regressor"`.

10.1 `run_from_experiment_file` — roteamento de configuração

L^a `configs/base.yaml` (defaults globais) e o YAML do experimento (Capítulo 13), combina os dois, e decide qual das três funções acima chamar, com base no *schema* do YAML:

- Se o YAML contém a chave `basis` (uma base) ou `all_bases: true` → experimento do FEMa (`run_basis_experiment` ou `run_all_bases`). A chave `context` é opcional, com *default* `"classifier"`.
- Se contém a chave `model` → baseline externo (`run_baseline_experiment`).

Isso significa que, na prática, o usuário quase nunca chama `run_basis_experiment` ou `run_baseline_experiment` diretamente — escreve um arquivo YAML pequeno (Capítulo 13) e roda via `main.py` (Capítulo 14).

11. Relatórios e comparação de bases: reporting/

11.1 compare_bases.py — o módulo ativo

Este é o módulo que materializa a pergunta de pesquisa do projeto: lê os `summary.json` e `test_results.json` de cada base já rodada (via `pipeline.run_model.run_basis_experiment/run_all_bases`) e produz, num único ponto de entrada:

```
from reporting.compare_bases import run_full_comparison

run_full_comparison(
    context="classifier", dataset="fetal_health", experiment_name="
        baseline",
    output_dir="reports/basis_comparison/classifier/fetal_health",
)
```

- `comparison_table.csv` — uma linha por base, uma coluna por métrica de teste (na ordem canônica de `metrics.registry.metrics_for_context`), mais a melhor combinação de hiperparâmetros de cada base.
- `comparison.json` — o mesmo conteúdo, em JSON, mais um `ranking_per_metric`: para cada métrica, as bases ordenadas da melhor para a pior (respeitando `HIGHER_IS_BETTER`).
- `plots/bar_<metrica>.png` — um gráfico de barras por métrica, todas as bases lado a lado, gerado para **toda** métrica registrada para o contexto que tenha ao menos um valor disponível.
- `plots/curve_<metrica>_vs_k.png` — uma curva de sensibilidade a k por base (uma linha por base), construída a partir do `ranking` de validação cruzada de cada `summary.json` — **não** do teste, já que o teste só produz um único ponto por base, insuficiente para uma curva. A própria docstring da função documenta uma limitação conhecida: como o *random search* varia o parâmetro próprio de cada base (`z`, `epsilon`, ...) simultaneamente a k , os pontos de uma mesma base podem não formar uma curva estritamente monotônica em k — o valor no eixo Y para um dado k é de **uma** combinação específica sorteada com aquele k , não uma média marginal sobre o parâmetro próprio da base. Para uma curva estritamente marginal em k , é preciso rodar um experimento com `fixed_hyperparameters` variando apenas k .

Os resultados de exemplo já presentes no repositório (`reports/basis_comparison/classifier/fetal_health`) foram gerados exatamente por esse fluxo, sobre o dataset `fetal_health`.

11.2 compare_models.py — arquivo legado, não utilizado

Este arquivo é uma versão **anterior** de `compare_bases.py` — de fato, sua própria docstring já afirma “*esta é a substituição de `reporting/compare_models.py` (removido)*”, mas o arquivo

antigo, na prática, não foi removido do repositório, apenas duplicado com o novo nome. `reporting/__init__.py` importa exclusivamente de `compare_bases`, então `compare_models.py` **não é executado por nenhum caminho de código ativo do projeto** atualmente. Recomenda-se removê-lo numa próxima limpeza, para não confundir alguém que o encontre e não perceba que está obsoleto (ver Capítulo 16).

11.3 `plots.py` e `curves.py` — utilitários de visualização gerais

Dois módulos independentes de `compare_bases.py`, com funções de plotagem mais genéricas (matriz de confusão, curva ROC/PR a partir de `y_score`, gráfico de valor previsto vs. real para regressão) e o cálculo manual (sem dependência além de *numpy*) de curvas ROC e precision-recall em `curves.py`. Não são chamados automaticamente por nenhum pipeline — existem como ferramentas prontas para uso pontual/exploratório (por exemplo, dentro de um notebook de análise).

12. Utilitários: `utils/`

- `logging_config.py::get_logger(name)` — logger único por módulo, formato padronizado, grava simultaneamente em `stdout` e em `logs/pipeline.log`. Evita handlers duplicados em reimportações.
- `seeding.py` — `set_global_seed(seed)` fixa as sementes de `random` e `numpy`; `derive_seed(master_seed, *parts)` deriva, via `hash` SHA-256 estável, uma seed determinística a partir de identificadores arbitrários (nome do plugin, `combo_id`, índice de fold, índice de repetição, ...). Isso garante que o **mesmo** fold/repetição sempre receba a **mesma** seed entre execuções diferentes — condição necessária tanto para reprodutibilidade quanto para o checkpoint.
- `timing.py::timer()` — *context manager* simples para medir tempo decorrido (usado para registrar `execution_time_seconds` em cada `run_*.json`).
- `hashing.py::stable_hash / run_identity` — geração de hashes SHA-256 determinísticos a partir de dicionários (canonicalizando chaves ordenadas e arredondando *floats* para evitar que 0.1 vs. 0.1000000000000001 gerem hashes diferentes). **Não é mais usado** pelo sistema de checkpoint atual (que usa `combo_id` + índices de fold/repetição diretamente, ver Capítulo 9) — é um remanescente de uma versão anterior do checkpoint, que identificava cada execução por um hash único. Mantido no repositório por ser potencialmente reaproveitável, mas hoje um código morto do ponto de vista do pipeline principal.
- `hardware_info.py::get_hardware_info / get_code_version` — coleta informação de plataforma, contagem de CPUs, disponibilidade de GPU/*CUDA* (via *torch*, se instalado) e o *commit git* atual. Também **não é chamado** por nenhum ponto do pipeline atual — outro remanescente disponível para uso futuro (por exemplo, anexar essas informações a cada `run_*.json`, para auditar em que máquina/versão do código cada resultado foi gerado).

13. Configuração declarativa: configs/

13.1 base.yaml — defaults globais

```
master_seed: 42
cross_validation:
  strategy: repeated_stratified_kfold
  n_splits: 5
  n_repeats: 3
tuning:
  strategy: random_search
  n_iter: 20
metrics: [accuracy, f1, balanced_accuracy, mcc]
ranking_metric: f1
higher_is_better: true
run_final_test: true
```

Todo campo aqui pode ser sobrescrito por um YAML de experimento individual.

13.2 experiments/*.yaml — um arquivo por experimento

Quatro exemplos já presentes ilustram os dois *schemas* de experimento reconhecidos por `run_from_experiment_file` (Capítulo 10):

```
# fema_baseline.yaml - roda TODAS as bases para um contexto/dataset
context: classifier
all_bases: true
dataset: fetal_health
experiment_name: baseline
tuning_strategy: random_search
tuning_n_iter: 20
```

```
# fema_shepard_only.yaml - escopado a UMA UNICA base
context: classifier
basis: shepard
dataset: fetal_health
experiment_name: baseline
```

```
# knn.yaml - baseline externo, fora da comparacao de bases
model: knn
dataset: fetal_health
experiment_name: baseline
```

`smoke_test.yaml` roda o baseline `logreg` sobre o dataset sintético, com validação cruzada reduzida (3 *folds*, 1 repetição) e apenas 3 iterações de *random search* — existe para validar rapidamente que o pipeline inteiro (do carregamento de dados à avaliação no teste) funciona de ponta a ponta, sem esperar o tempo de um experimento real.

14. Ponto de entrada: main.py

A interface de linha de comando cobre quatro modos de uso, que podem ser combinados:

```
# Roda um ou mais experimentos a partir de arquivo(s) YAML
python main.py --experiment configs/experiments/fema_baseline.yaml

# Roda todas as bases diretamente por flags (sem YAML)
python main.py --context classifier --all-bases --dataset fetal_health

# Roda uma unica base
python main.py --context classifier --basis shepard --dataset
    fetal_health

# Aciona a comparacao entre bases ja rodadas (nao roda experimentos)
python main.py --compare --context classifier --dataset fetal_health

# Roda todas as bases E compara em seguida, num unico comando
python main.py --context classifier --all-bases --dataset fetal_health
    --compare

# Baseline externo
python main.py --baseline-model knn --dataset fetal_health
```

A flag `--compare` não roda nenhum experimento por si só — ela lê os resultados já persistidos em `results/<context>/<basis>/<dataset>/<experiment_name>/` e gera a tabela, o JSON consolidado e os gráficos via `reporting.compare_bases.run_full_comparison` (Capítulo 11). Pode ser combinada com `--all-bases` para, num único comando, rodar todas as bases e já produzir a comparação em seguida.

15. Testes: `tests/`

15.1 `test_pipeline.py` — testes de fumaça do pipeline

Usa o dataset sintético (rápido, sem depender de arquivos externos) para validar, de ponta a ponta: que o FEMa treina e prediz corretamente (`test_fema_classifier_end_to_end`, com um exemplo geometricamente óbvio — dois agrupamentos bem separados); que `run_basis_experiment` persiste no caminho correto, `results/<context>/<basis>/<dataset>/<experiment_name>/`; que `run_baseline_experiment` persiste isolado em `results/external_baselines/...`; e que o checkpoint efetivamente evita duplicar execuções (roda o mesmo experimento duas vezes e confere que o número de arquivos `run_*.json` não muda na segunda vez).

15.2 `test_reporting.py` — testes do módulo de comparação

Roda `run_all_bases` sobre um pequeno subconjunto de bases (`shepard`, `radial`, `rbf_gaussian`) no dataset sintético, usando **todas** as métricas registradas para o contexto (não um subconjunto reduzido), e confere que: cada linha da comparação traz todas as métricas esperadas; o JSON consolidado e o CSV contêm exatamente essas métricas; e `run_full_comparison` gera um gráfico de barras e uma curva de k para **cada** métrica registrada — nenhuma a mais, nenhuma a menos.

15.3 `tests/linear_algebra/` — validação matemática das bases

Dois arquivos com sobreposição significativa de conteúdo: `math_validation.py` (mais antigo) e `validation.py` (mais recente e mais completo — ver Capítulo 16). Ambos implementam um *harness* que verifica propriedades matemáticas de cada base registrada em `Basis.available()`, automaticamente (sem precisar listar as bases manualmente): sanidade estrutural (não levantar exceção nem retornar NaN), partição da unidade ($\sum w_i = 1$), não-negatividade, igualdade de pesos para distâncias iguais, monotonicidade, e uma métrica específica do domínio chamada *leakage* (vazamento de interpolação).

A ideia de *leakage* merece destaque por ser conceitualmente central à validação: o que importa para a qualidade da interpolação não é o vetor de pesos em si comparado a um “vetor ideal” $[1, 0, 0, \dots]$, mas sim o quanto a predição final **pode** se afastar do valor verdadeiro no pior caso. Com partição da unidade e o vizinho de distância zero no índice j :

$$\text{predição} - y_j = \sum_i w_i (y_i - y_j) \quad \Rightarrow \quad |\text{predição} - y_j| \leq (1 - w_j) \max_i |y_i - y_j|.$$

O termo $(1 - w_j)$ — a soma dos pesos em todos os vizinhos **exceto** o de distância zero — é o *leakage*: simultaneamente o limite superior do erro de interpolação e um valor atingível. A versão mais recente (`validation.py`) vai além da dedução algébrica e verifica isso **empiricamente** (`check_leakage_matches_worst_case_prediction`): monta um caso sintético de

pior caso ($y_j = 0$, $y_i = 1$ nos demais) e confere que a predição real bate com a fórmula fechada do *leakage*.

`validation.py` também inclui `SWEEP_CONFIG`: para cada base, qual campo de `BasisParameters` é o “parâmetro de escala” (o que controla a concentração da função perto de $d = 0$) e em que faixa varrê-lo, permitindo auditar como o *leakage* muda conforme esse parâmetro varia — inclusive documentando, na própria configuração, os casos em que uma base **não tem** hoje um parâmetro de escala (`lorentzian`, `attention`), o que é descrito como uma limitação real da fórmula atual, não uma omissão do harness.

16. Código legado e pendências

Para que este documento seja útil como mapa fiel do repositório (e não apenas da arquitetura “ideal”), vale registrar explicitamente o que existe no código hoje mas não faz parte do caminho ativo:

- **src/** — a árvore de módulos anterior à arquitetura atual (`config.py`, `cross_validation.py`, `curves.py`, `datasets.py`, `metrics.py`, `models.py`, `persistence.py`, `pipeline.py`, `plots.py`, `tuning.py`, `utils.py`). Nenhum módulo fora de **src/** importa nada dela. É segura para remoção; foi mantida até aqui apenas como referência histórica de como o projeto era organizado antes da divisão em camadas descrita neste documento.
- **reporting/compare_models.py** — versão anterior de `compare_bases.py` (Seção 11), não referenciada por `reporting/__init__.py` nem por nenhum outro módulo. Candidato a remoção.
- **models/mlp.py** — implementado, mas não registrado em `models/registry.py::_PLUGINS`. Re-habilitá-lo é uma mudança de uma linha (ver Seção 17.4).
- **utils/hashing.py** e **utils/hardware_info.py** — utilitários funcionais, mas não chamados por nenhum ponto do pipeline atual (o checkpoint usa `combo_id` + índices de fold/repetição, não hashes; nenhum `run_*.json` grava informação de hardware hoje). Ficam disponíveis para uso futuro.
- **tests/linear_algebra/math_validation.py** — sobreposto por `validation.py` (mais completo, com verificação empírica de *leakage* e `SWEEP_CONFIG`). Mantido, mas duplica boa parte do conteúdo do arquivo mais novo.
- **teste.ipynb** — notebook de uso exploratório, fora do escopo desta documentação (não faz parte do pipeline reproduzível).

Nenhum destes pontos é urgente — o pipeline funciona corretamente com eles presentes —, mas vale uma limpeza numa próxima iteração, para reduzir a chance de alguém editar por engano um arquivo que não é mais executado.

17. Como expandir o projeto

Este capítulo é o mais orientado à ação: cobre, com passos concretos, as quatro direções de expansão mais prováveis para este projeto. Em todos os casos, a resposta curta é a mesma — **a arquitetura em camadas foi desenhada exatamente para que essas expansões não exijam tocar em core/, pipeline/, persistence/, training/ ou evaluation/**. Cada extensão se resume a adicionar um arquivo pequeno (ou uma entrada em um dicionário) na camada correspondente.

17.1 Adicionando um novo dataset

Passos:

1. Coloque o CSV em data/raw/.
2. Adicione uma entrada em datasets/registry.py::_REGISTRY:

```
"meu_dataset": DatasetSpec(  
    name="meu_dataset",  
    csv_path=DATA_RAW / "meu_arquivo.csv",  
    target_column="minha_coluna_alvo",  
    drop_columns=("id",),          # opcional  
    delimiter=";",                # opcional, default ja e' ", "  
)
```

3. Pronto — `datasets.loader.load_dataset("meu_dataset")` já funciona, com split treino/validação/teste determinístico e padronização automática. Nenhum outro arquivo precisa ser tocado: qualquer experimento (FEMa ou baseline) passa a poder usar `dataset: meu_dataset` no YAML (Capítulo 13).

Se o alvo for **contínuo** (dataset de regressão), não é preciso nenhum passo adicional aqui — `loader.py` já detecta automaticamente se a coluna-alvo é numérica ou categórica. A diferenciação entre classificação e regressão acontece na escolha do `context` ao rodar o experimento (próxima seção), não no cadastro do dataset.

17.2 Adicionando uma nova base de interpolação ao FEMa

Este é o fluxo de expansão mais frequente do projeto, e o mais bem pavimentado — a própria existência de `Basis.available()` como fonte única de verdade (Seção 2.3.5) foi desenhada para isso.

1. Crie `core/math/basis/minha_base.py`:

```
import numpy as np  
from .base_basis import BaseBasis, DELTA
```

```

from .parameters import BasisParameters

class MinhaBase(BaseBasis):
    """phi(d) = ... (descreva a formula e cite a referencia, se
    houver)."""
    PARAMS = ("epsilon",) # os campos de BasisParameters que a
    formula usa

    def evaluate(self, dists: np.ndarray, params: BasisParameters)
    -> np.ndarray:
        epsilon = self._require(params)["epsilon"]
        dists = np.where(dists == 0, DELTA, dists) # se houver
        risco de divisao por zero
        return ... # a formula em si

```

Não implemente `compute_weights` — a normalização já está pronta em `BaseBasis` (Seção 2.3).

2. Registre em `core/math/basis/factory_basis.py`:

```

from .minha_base import MinhaBase
_CLASSES = {
    ...,
    "minha_base": MinhaBase,
}

```

3. Adicione a entrada correspondente em `core/math/basis/__init__.py` (`import + __all__`), para consistência com as demais bases.
4. Em `models/fema.py`, adicione a faixa de busca de hiperparâmetros em **ambos** `PARAM_GRIDS` e `RANDOM_SEARCH_SPACES`:

```

PARAM_GRIDS["minha_base"] = {"epsilon": [0.1, 1.0, 5.0, 20.0]}
RANDOM_SEARCH_SPACES["minha_base"] = {"epsilon": ("loguniform",
0.1, 50.0)}

```

Se esquecer este passo, `_validate_basis_entry` levanta um `ValueError` claro assim que alguém tentar instanciar `FEMaPlugin(context=..., basis="minha_base")` — o erro aparece na hora certa (construção do plugin), não no meio de um tuning de horas.

5. (Recomendado) Adicione uma entrada em `SWEEP_CONFIG`, em `tests/linear_algebra/validation.py`, para que a nova base seja incluída automaticamente na varredura de validação matemática (partição da unidade, monotonicidade, *leakage*, ... — Capítulo 16).

A partir daqui, a nova base já aparece automaticamente em `Basis.available()`, e portanto: já pode ser usada em `basis: minha_base` num YAML de experimento (Seção 4.2); já é incluída quando alguém roda `all_bases: true`; e já aparece automaticamente nas tabelas e gráficos de `reporting/compare_bases.py` (Capítulo 11) — nenhum desses três lugares precisa ser editado manualmente.

17.3 Habilitando análise de regressão

Este é talvez o ponto mais importante deste capítulo: **a análise de regressão já está implementada** — `FEMaRegressor` existe em `core/models/fema_regressor.py` desde a base do projeto, `FEMaPlugin` já aceita `context="regressor"` (Seção 4.2), e `metrics/regression.py`

já define as cinco métricas de regressão (`mae`, `mse`, `rmse`, `r2`, `mape`). O que falta para rodar um experimento de regressão de ponta a ponta é apenas:

1. Um dataset com alvo **contínuo** registrado em `datasets/registry.py` (Seção 17.1) — nenhum dos quatro datasets atuais tem alvo contínuo, então este é o único passo genuinamente necessário.
2. Rodar (ou escrever um YAML) com `context: regressor`:

```
from pipeline.run_model import run_all_bases
run_all_bases(context="regressor", dataset="meu_dataset_continuo")
```

ou, via YAML:

```
context: regressor
all_bases: true
dataset: meu_dataset_continuo
experiment_name: baseline
```

A partir daí, todo o resto do pipeline já se adapta automaticamente: `run_basis_experiment` já troca as métricas e a métrica de *ranking* padrão para as de regressão (`DEFAULT_REGRESSION_METRICS`, `ranking_metric="rmse"` — Capítulo 10); `_FEMaEstimator` já instancia `FEMaRegressor` em vez de `FEMaClassifier` (Seção 4.2); `evaluate_fold/evaluate_on_test` já chamam `metrics.registry.comp` com as métricas de regressão corretas; e os resultados vão para `results/regressor/<basis>/<dataset>/<exp>` paralelo à árvore de classificação, sem colidir com ela. `reporting.compare_bases.run_full_comparison(context="regressor", ...)` já produz a tabela e os gráficos comparando as bases também neste contexto.

Em suma: nenhuma linha de `core/`, `pipeline/`, `metrics/`, `models/` ou `reporting/` precisa ser alterada para habilitar regressão — a única lacuna real é a ausência, hoje, de um dataset de regressão cadastrado.

17.4 Comparando com outros modelos (baselines)

Existem dois casos distintos aqui, que vale diferenciar:

17.4.1 Re-habilitar um baseline já implementado (MLP)

Como observado no Capítulo 16, `models/mlp.py` já existe e é funcional — só não está no registro. Basta:

```
# models/registry.py
from models.knn import KNNPlugin
from models.logreg import LogRegPlugin
from models.mlp import MLPPlugin          # adicionar este import

_PLUGINS = {
    "logreg": LogRegPlugin(),
    "knn": KNNPlugin(),
    "mlp": MLPPlugin(),                   # adicionar esta linha
}
```

A partir daí, `model: mlp` já funciona em qualquer YAML de experimento, exatamente como `logreg/knn`.

17.4.2 Adicionar um baseline totalmente novo

Passos, usando como exemplo um `RandomForestClassifier` do scikit-learn:

1. Crie `models/random_forest.py`:

```
from typing import Any, Dict
from sklearn.ensemble import RandomForestClassifier
from models.base import ModelPlugin

class RandomForestPlugin(ModelPlugin):
    name = "random_forest"
    supports_proba = True

    def create_model(self, params: Dict[str, Any], random_state:
int):
        return RandomForestClassifier(
            n_estimators=params.get("n_estimators", 200),
            max_depth=params.get("max_depth", None),
            random_state=random_state,
        )

    def parameter_grid(self) -> Dict[str, list]:
        return {"n_estimators": [100, 200, 400], "max_depth": [
            None, 5, 10, 20]}

    def random_search_space(self) -> Dict[str, Any]:
        return {"n_estimators": ("randint", 50, 500), "max_depth":
            [None, 5, 10, 20, 40]}
```

2. Registre em `models/registry.py::_PLUGINS`, como acima.
3. Crie um YAML em `configs/experiments/` com `model: random_forest`, ou chame `pipeline.run_model.run_baseline_experiment(model_name="random_forest", dataset=...)` diretamente.

Nenhuma outra camada precisa mudar: `training/`, `evaluation/`, `tuning/` e `persistence/` já funcionam para qualquer `ModelPlugin`, seja ele o FEMa ou um baseline externo. A única diferença estrutural entre um baseline e o FEMa é que baselines usam um único nome fixo (`model_name`) no registro, enquanto o FEMa é parametrizado por (`context`, `basis`) — mas isso é transparente para quem só quer adicionar *mais um* baseline de comparação.

Para incluir o novo baseline na mesma tabela comparativa das bases do FEMa, basta chamar `reporting.compare_bases.compare` para o baseline separadamente (ele fica em `results/external_baseline`) e concatenar as linhas manualmente, ou escrever uma pequena função de comparação combinada — isso é o único ponto em que a separação deliberada entre “comparação de bases” e “baselines externos” (Seção 11) exige um passo manual extra, por design: o projeto não quer que um baseline externo apareça, por acidente, misturado no ranking de bases do FEMa.

17.5 Criando um novo experimento (YAML)

Para qualquer combinação já suportada pelo código (`dataset × contexto × base`, ou `dataset × baseline`), criar um novo experimento é só escrever um YAML novo em `configs/experiments/`, seguindo um dos dois *schemas* da Seção 13 — nunca é necessário editar `pipeline/run_model.py` para isso. Campos omitidos herdam de `configs/base.yaml`.

17.6 Adicionando uma nova métrica

1. Escreva a função em `metrics/classification.py` ou `metrics/regression.py`, seguindo o contrato de `metrics/contracts.py` (`metric(y_true, y_pred, y_score, n_classes)` -> `Optional[float]`).
2. Registre em `metrics/registry.py`: uma entrada em `_METRICS`, uma em `HIGHER_IS_BETTER`, e inclua o nome em `CLASSIFICATION_METRICS` ou `REGRESSION_METRICS`, conforme o caso.

A partir daí, a métrica passa a estar disponível para `metric_names` em qualquer YAML de experimento, para `ranking_metric`, e é automaticamente incluída nas tabelas e nos gráficos de `reporting/compare_bases.py` — nenhum desses três consumidores precisa ser editado.

17.7 Checklist resumo

Quero...	Arquivo(s) a tocar
Novo dataset	<code>datasets/registry.py</code> (uma entrada)
Nova base do FEMA	novo arquivo em <code>core/math/basis/</code> + <code>factory_basis.py</code> + <code>core/math/basis/__init__.py</code> + <code>models/fema.py</code> (<code>PARAM_GRIDS</code> e <code>RANDOM_SEARCH_SPACES</code>)
Análise de regressão	apenas um dataset de alvo contínuo (Seção 17.1) + <code>context: regressor</code> na chamada/YAML
Re-habilitar o MLP	<code>models/registry.py</code> (duas linhas)
Novo baseline	novo arquivo em <code>models/</code> + <code>models/registry.py</code>
Novo experimento	novo YAML em <code>configs/experiments/</code>
Nova métrica	função em <code>metrics/classification.py</code> ou <code>regression.py</code> + <code>metrics/registry.py</code>

18. Conclusão

A arquitetura descrita neste documento é o resultado de uma decisão de design consistente, repetida em cada camada: **isolar o que muda (experimentos, datasets, bases, métricas) do que não muda (a matemática em `core/`, o motor de execução em `pipeline/run_model.py`)**. Isso é o que torna o projeto extensível na prática, e não apenas em teoria — como ilustrado pelo Capítulo 17, cada uma das direções de expansão mais prováveis (novo dataset, nova base, regressão, novo baseline) se resume a um arquivo pequeno ou a uma entrada em um dicionário, nunca a uma reescrita de camadas já existentes.